LLMagikarp: a Large Language Model agent for Pokémon Battles

Garrett Herb
The Ohio State University
herb.45@osu.edu

Abstract

This paper presents LLMagikarp, a large language model (LLM) based agent for playing competitive Pokémon battles. Building upon the PokéLLMon framework, I implement and evaluate several novel augmentations to improve strategic decision-making, including a memory system for tracking battle history, opposition modeling for anticipating opponent moves, and initial strategy planning for team composition analysis. Through extensive testing against both heuristic bots and human players on the Pokémon Showdown platform, I evaluate these augmentations using GPT-4o and GPT-4o-Mini models. My results demonstrate that while the memory-based implementation achieved the highest local win rates (54.5%) with GPT-4o, all implementations faced persistent challenges including conservative switching behavior, vulnerability to setup strategies, and limitations in state representation. The GPT-4o model consistently outperformed its Mini counterpart, achieving higher consensus rates (88-90.5% vs 72.8-75.7%) and more coherent decision-making patterns. However, this improved consistency often came at the cost of reduced strategic adaptability. My findings reveal fundamental tensions between strategic consistency and tactical flexibility in LLM-based game agents, suggesting the need for hybrid architectures that combine LLM reasoning with specialized battle analysis tools and adaptive learning mechanisms. This research contributes to our understanding of LLM capabilities in complex strategic environments and provides insights for developing more effective game-playing agents.

1 Introduction

Although large language models (LLMs) have demonstrated remarkable capabilities across various domains, developing complex game strategies remains a significant challenge (Wang, L. et al., 2023). Understanding how LLMs approach strategic reasoning in games is crucial for several reasons. First, game environments provide controlled settings where we can systematically evaluate an LLM's ability to plan, reason, and adapt skills that are essential for more general AI applications. Second, games provide an accessible benchmark for measuring progress in AI reasoning capabilities, as performance can be directly quantified through win rates and strategy effectiveness.

Pokémon battles offer an ideal test environment for exploring these questions due to their vast action space, requirement for multi-turn strategic planning, and need for dynamic opponent adaptation (Hu, S. et al., 2024). The complexity of these battles comes from their expansive possibilities: 1,025 different Pokémon species, 934 possible moves, and 18 elemental types, creating an enormous combinatorial space for strategic decision-making. Each battle requires both immediate tactical choices and longer-term strategic planning, making it an excellent benchmark for evaluating AI agents' reasoning capabilities.

Building upon the PokéLLMon framework (Hu S. et al., 2024), this project looks to validate their findings and also implement and evaluate several novel augmentations:

- A memory system that maintains and utilizes information about past decisions and their outcomes
- Opposition modeling capabilities that analyze potential opponent moves and their consequences
- Initial strategy planning that evaluates team composition and move set synergies

Through extensive testing against both heuristic bots and human players on the Pokémon Showdown platform, we identify several key challenges in LLM-based game agents, including conservative switching behavior, vulnerability to setup strategies, and limitations in state representation. These

findings not only contribute to the development of better game-playing agents but also provide broader insights into the capabilities and limitations of LLMs in complex reasoning tasks.

2 Related Work

Recent work in LLM-based game agents has demonstrated both the potential and limitations of using language models for strategic decision-making. LLM agents have been successfully deployed across various game environments, from text-based adventures to complex strategy games (Hu et al., 2024). The PokéLLMon framework specifically showed promising results in Pokémon battles, achieving competitive performance against human players through text-based state representation and decision-making.

The challenge of maintaining consistent strategy across multiple turns has been addressed through various approaches in the literature. Wang, G. et al. (2023) demonstrated the effectiveness of skill libraries for continuous learning in game environments, while others have explored the use of memory mechanisms to maintain strategic coherence (Lewis et al., 2020).

A particular focus has been placed on the balance between model size and performance. While larger models like GPT-4 typically demonstrate superior reasoning capabilities, recent work has shown that smaller, specialized models can achieve comparable results in specific domains through appropriate fine-tuning and augmentation (Liu, S. et al., 2024).

3 Pokémon

The popular franchise Pokémon centers around a series of video games where players explore a fantasy world where monsters, or Pokémon, are able to be captured and trained. To progress in the mainline games, the player must battle other 'trainers.' This battling mechanism has become a huge cultural sensation with many events pinning players against each other for formal competitions. While the premise of the battle is simple, eliminate your opponent's Pokémon before your own are eliminated, the battling is extremely complex and requires players to consider a huge amount of information and possibilities.

While many battling formats exist, the agent will only be employed in generation 9 random single battles. This adheres to the traditional single battle format of the Pokémon games while exposing the agent to much more diverse starting states from the teams being randomized. In random single battles, each player is given 6 random Pokémon, where only one can be active at each time. For each turn, both players are able to choose from a couple action choices. They can either choose one of 4 moves the active Pokémon has access to or switch the active Pokémon with an unactive Pokémon that has not been knocked out yet. Both players pick actions in order to reduce the health of the opposing Pokémon to knock them out and to keep their own Pokémon from accruing damage. Again, this premise seems simple, but there are many variables that contribute to a game where the next state is impossible to consistently predict.

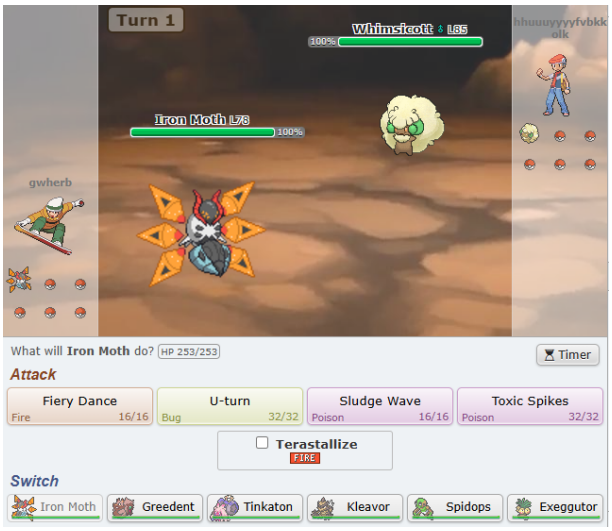

Figure 1: Screenshot from the Pokémon Showdown Battling Simulator

3.1 Pokémon species

Each of the 6 random Pokémon on a given team are one of 1,025 unique species all with attributes that pertain to that specific species (Bulbapedia, 2024b).

3.2 Moves

In a battle, all Pokémon have 4 moves. Moves are meant to either damage a targeted Pokémon, inflict a negative status to a target Pokémon, inflict a positive status on the targeted Pokémon, or heal a targeted Pokémon. While there are only 4 moves per Pokémon in the battles, there exist 934 unique moves that Pokémon are able to learn (Bulbapedia, 2024b).

3.3 Types

There exist 18 elemental types where each Pokémon can have either 1 type or a combination of 2 types. Each move has a single elemental type as well. Every type has certain weaknesses and advantages against other types (Bulbapedia, 2024b).

3.4 Abilities

Every Pokémon has 1 to 4 innate "abilities" that can alter the effect of opposing moves on itself or affect its own attacking moves (Bulbapedia, 2024b).

3.5 Stats and Nature

All Pokémon have 6 stat categories (Health, Attack, Defense, Special Attack, Special Defense, and Speed). Each stat can be balanced to allow Pokémon to be faster, stronger attackers, stronger defenders, etc. All Pokémon have their own initial spread of these stats that can also be augmented by Pokémon natures. Each Pokémon can be one of 25 natures that correspond to a change in initial stats (Bulbapedia, 2024b).

4 Method

My approach builds upon the PokéLLMon framework while introducing several novel augmentations designed to enhance strategic decision-making. This section details the base architecture and specific augmentations implemented.

4.1 Base Framework

The foundation of our system follows the text-based perception approach established in PokéLLMon (Hu et al., 2024) which is visualized in Figure 2. The battle state is translated into a textual format that captures all relevant information, including:

- Current Pokémon statistics and status
- Available moves and their properties
- Team composition and health states
- Previous turn outcomes

All LLM prompts utilize three examples (3-shot) with chain of thought (Wei, J. et al., 2022). Two different language models are evaluated in our implementation:

- GPT-4o: A larger model with demonstrated capabilities in complex reasoning
- GPT-4o-Mini: A smaller model to evaluate performance trade-offs

4.2 Memory Capabilities

One key limitation of the original PokéLLMon framework was its lack of access to past decision reasoning. This is addressed through a memory system that stores and integrates:

- Past two move decisions and their rationale
- Results and effectiveness of those moves
- Integration of this historical information into the current battle state

This memory system enables the agent to develop and maintain multi-turn strategies while learning from the immediate outcomes of its decisions. The historical context is provided to the model along with the current state, allowing for more coherent gameplay planning.

4.3 Opposition Modeling

To enhance the strategic depth, opposition modeling capabilities were implemented. While the original framework only considered the agent's possible actions, our system analyzes:

- Opponent's potential moves and their probabilities
- Likely consequences of opponent actions
- Possible team composition strategies

This is achieved through enhanced prompt engineering that explicitly asks the model to consider opponent behavior before making decisions. The opposition modeling component aims to make decisions more strategically sound by anticipating and countering opponent actions.

4.4 Initial Strategy Planning

To address the challenge of random team assignments leading to incoherent choice of movements, we implemented an initial strategy planning system. At the start of each battle, the system evaluates:

- Individual Pokémon strengths and weaknesses
- Move set synergies within the team
- Potential team-wide strategic opportunities

This initial analysis helps guide move selection throughout the battle, attempting to maintain strategic coherence despite the randomized nature of team composition.

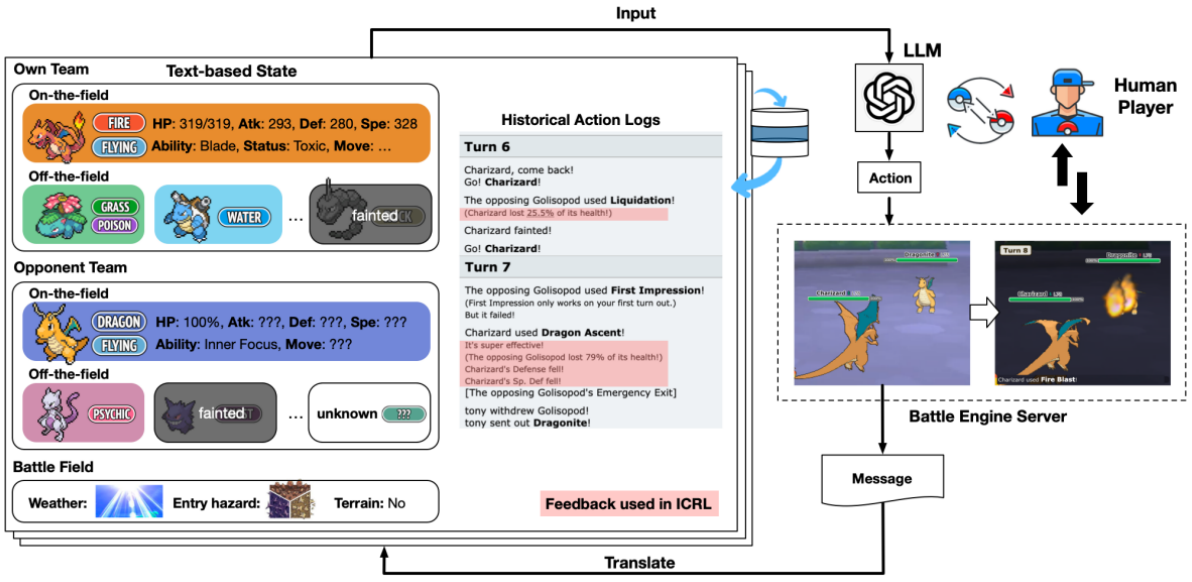

Figure 2: Base Framework for PokéLLMon Implementation

4.5 Model Variants and Training

We tested several variants of our system to evaluate the impact of different components:

- Base Player: Basic implementation of the original framework (Figure 2)
- Self-Consistency Player: Re-implementation of a player from the PokéLLMon equipped with self-consistency (Wang, X. et al., 2022)
- Memory Player: Enhanced with memory capabilities
- Opposition Player: Added opposition modeling
- Initial Strategy Player: Includes initial strategy planning

Each variant was tested against both a heuristic bot and human players on the Pokémon Showdown ladder, allowing for a comprehensive evaluation of each augmentation's impact.

5 Results

Comprehensive stats were taken for every augmentation using both GPT-4o and GPT-4o-Mini. Here, local battles refer to the matches against the heuristic bot and ladder refers to matches against humans. The results reveal significant differences between model variants and highlight several key challenges in LLM-based game agents.

5.1 Model Performance

The choice of base model significantly impacted performance across all implementations. GPT-4o consistently outperformed GPT-4o-Mini, demonstrating both higher win rates and more coherent decision-making patterns. While GPT-4o showed the ability to leverage built-in Pokémon knowledge for informed decisions, GPT-4o-Mini struggled with the complex state representations, leading to higher hallucination rates and poor move selection. The fluctuation in the win rate for models against humans on the ladder are a result of human behavior. Humans tended to forfeit matches when found in a slightly disadvantageous situation resulting in boosted win rates for some models that happened to make a good early turn. Additionally, while the ladder does have skill-based match making, not every human at a given rank is of equal skill.

Model	Local		Ladder	
	GPT-4o	Mini	GPT-4o	Mini
Base	45.5%	6.2%	33.3%	16.0%
SC3	40.0%	5.0%	48.0%	37.5%
Memory	54.5%	6.0%	50.0%	16.2%
Opposition	50.0%	8.0%	50.0%	23.1%
Initial Strategy	20.0%	4.0%	36.4%	12.0%

Table 1: Win Rates by Model and Implementation

5.2 Battle Statistics and Behavioral Patterns

Analysis of battle statistics revealed a persistent pattern of conservative switching behavior across all implementations. This manifested in high double-switch rates, particularly in Mini variants (10.9-14.5%), often creating passive gameplay loops that reduced strategic effectiveness. GPT-4o models achieved shorter average game lengths (20.4-32.0

Model	Avg. Turns	Switch Rate	Double Switch Rate
Base (GPT-4o)	20.4	35.5%	9.6%
SC3 (GPT-4o)	26.3	36.1%	7.2%
Memory (GPT-4o)	22.2	30.5%	4.7%
Opposition (GPT-4o)	22.2	39.2%	8.6%
Initial Strategy (GPT-4o)	32.0	32.5%	11.3%
Base (Mini)	28.1	37.1%	14.5%
SC3 (Mini)	26.9	36.6%	11.0%
Memory (Mini)	27.7	31.5%	10.9%
Opposition (Mini)	28.9	37.1%	13.2%
Initial Strategy (Mini)	28.4	31.7%	12.4%

Table 2: Battle Duration and Switching Patterns by Implementation

turns) compared to Mini variants (26.9-28.9 turns), suggesting more decisive gameplay.

While switching one's Pokémon into a benched Pokémon to have a more favorable matchup, frequent switching can be harmful. When switching out frequently, the player is not doing any damage while likely taking significant damage in return. This leads to negative progress which almost always will lose the game for the player.

5.3 Implementation Effectiveness

Model	Total Defeats (% of Games)	Close Losses (% of Games)
Base (GPT-4o)	19.51	17.07
SC3 (GPT-4o)	7.32	19.51
Memory (GPT-4o)	0.00	24.14
Opposition (GPT-4o)	6.67	13.33
Initial Strategy (GPT-4o)	4.76	42.86
Base (Mini)	13.59	20.63
SC3 (Mini)	11.64	17.12
Memory (Mini)	14.94	22.99
Opposition (Mini)	7.94	25.40
Initial Strategy (Mini)	28.30	11.32

Table 3: Game Outcomes as Percentage of Total Games

The rates of total defeats versus close losses give further granularity on the manner in which the models lost. With win rates all around the same level, this allows better insight into performance among each other. Here, total defeats refer to games played where the opposing team won with 5 or more Pokémon remaining. Close losses refer to games that were lost with 2 or less Pokémon remaining.

We see that all augmentations to the base model made improvements when using the GPT-4o model, however, there was no significant change between the players, apart from the initial strategy player, when using the Mini model. This shows that implementing augmentations via prompt engineering that increase the context size end up being at best neutral and at worst harmful to Mini-model agents.

5.4 Self-Consistency Analysis

Model	Consensus Rate	Consensus Turns	Total Turns
GPT-4o Local	90.5%	238	263
GPT-4o Ladder	88.2%	365	414
Mini Local	75.7%	2096	2767
Mini Ladder	72.8%	669	919

Table 4: Self-Consistency Performance Metrics

The self-consistency implementation revealed interesting trade-offs between stability and adaptability. The consensus rate refers to the percent of turns where the chosen move was decided by at least 2 of the 3 move generations. GPT-4o achieved significantly higher consensus rates (88-90.5%) compared to Mini variants (72.8-75.7%). However, this consistency came at the cost of reduced move variety and potential predictability.

5.5 Decision-Making Patterns

Model	Attack Rate	Switch Rate	Random Rate
Base (GPT-4o)	61.5%	38.5%	4.1%
SC3 (GPT-4o)	63.5%	36.5%	2.1%
Memory (GPT-4o)	69.6%	30.4%	3.5%
Opposition (GPT-4o)	64.2%	35.8%	1.3%
Initial Strategy (GPT-4o)	65.7%	34.3%	5.1%
Base (Mini)	62.9%	37.1%	8.9%
SC3 (Mini)	65.4%	34.6%	9.6%
Memory (Mini)	68.6%	31.4%	9.8%
Opposition (Mini)	65.6%	34.4%	7.5%
Initial Strategy (Mini)	67.1%	32.9%	11.0%

Table 5: Decision-Making Patterns by Implementation

Analysis of decision-making patterns revealed complex trade-offs between different implementations. While GPT-4o and Mini variants showed similar attack-to-switch ratios (around 65% attacks to 35% switches), there were notable differences in decision validity. A critical limitation emerged in the form of random moves - instances where the models generated actions that weren't actually available in the game state and a random action was chosen instead. The GPT-4o implementations demonstrated better adherence to valid move sets, with Opposition modeling showing the lowest random move rate (1.3%) and Initial Strategy the highest (5.1%). However, all Mini variants struggled significantly with move validity, generating invalid moves at roughly double the rate of their GPT-4o counterparts (7.5-11.0%).

This disparity in random move generation suggests that model size significantly impacts the ability to understand and respect game constraints. Memory-based implementations, despite showing the highest attack rates (69.6% for GPT-4o, 68.6% for Mini), couldn't overcome this limitation in the Mini variant, with invalid moves occurring in 9.8% of decisions. The SC3 approach demonstrated an interesting pattern - while it achieved the second-lowest random move rate in GPT-4o (2.1%), its Mini variant still generated invalid moves 9.6% of the time, suggesting that self-consistency checks alone cannot fully address move validity issues in smaller models. These patterns indicate that while different augmentations can maintain consistent action distributions, they fundamentally struggle with action space constraints in smaller models. The results support exploring hybrid approaches that could incorporate explicit move validation systems while maintaining the strategic reasoning capabilities observed in the larger models.

5.6 Conclusions and Implications

The comprehensive analysis of LLMagikarp's performance reveals several key insights about LLM-based game agents and the effectiveness of various augmentation strategies.

5.7 Key Performance Insights

Significant patterns across implementations were observed that illuminate both the potential and limitations of LLM-based game agents. The memory-based implementation proved most successful in local testing, achieving a 54.5% win rate with GPT-4o and demonstrating the most aggressive attack patterns (69.5% attack rate). However, this performance did not fully translate to ladder play, where self-consistency mechanisms showed stronger results with a 48% win rate against human players. This disparity between local and ladder performance suggests that success against deterministic opponents may not predict effectiveness in dynamic human matchups.

Implementation complexity showed diminishing returns, with simpler augmentations often outperforming more sophisticated approaches. For instance, opposition modeling maintained consistent 50% win rates in ladder play despite its complexity, while the theoretically promising initial strategy planning approach proved least effective with only 20% win rates in local testing. Random move rates also increased with implementation complexity, ranging from 1.3% in basic implementations to 11% in more complex variants, suggesting that additional features may strain the model's ability to maintain move validity.

The trade-off between consistency and adaptability emerged as a central theme across all implementations. Higher self-consistency rates (88-90.5%) correlated with improved performance against predictable opponents but showed limitations against adaptive human players. Conversely, implementations with lower consistency rates demonstrated greater tactical flexibility but suffered from increased invalid move generation and strategic inconsistency. These patterns suggest that effective game agents may require a careful balance between strategic coherence and adaptive capability.

5.8 Persistent Technical Challenges

Several persistent challenges emerged across all implementations, each representing significant hurdles for LLM-based game agents:

5.8.1 Conservative Switching Behavior:

The observed switch rates of 30-39% across implementations revealed a fundamental tendency toward overly cautious play. This manifested in frequent defensive switching patterns where agents would repeatedly cycle through their team rather than commit to offensive actions. The double-switch rate, reaching as high as 14.5% in Mini variants, created passive gameplay loops that prevented the agent from maintaining offensive pressure. Even attempts to modify the prompt with more aggressive language failed to significantly alter this behavior, suggesting this conservatism might be inherent to the LLM's risk assessment approach.

5.8.2 Vulnerability to Setup Strategies:

The agents demonstrated a consistent inability to properly counter setup-based strategies, where opponents forgo doing damage to use a move that will boost the stats of the active Pokémon. This susceptibility persisted across all augmentations, with opposition modeling only providing marginal improvements. Analysis of game logs revealed that agents would often continue with standard attack patterns even after opponents had accumulated multiple stat boosts, leading to complete team sweeps. This suggests a fundamental limitation in the models' ability to recognize and respond to long-term threat development.

5.8.3 State Representation Overhead:

The translation of battle states into text created substantial computational and performance challenges. For GPT-4o-Mini, this resulted in a 72.8-75.7% self-consistency rate compared to GPT-4o's 88-90.5%, indicating frequent misinterpretation of game states. The problem was compounded by the need to include previous states for temporal context, creating increasingly lengthy inputs that degraded model performance. This was particularly problematic in cases involving status conditions and field effects, where complete state description required extensive text.

5.8.4 Strategy-Adaptability Trade-off:

My attempts to improve strategic consistency through augmentations like self-consistency checking and initial strategy planning revealed a fundamental tension in the system. Higher consistency rates (88-90.5% in GPT-4o) correlated with improved performance against the heuristic bot but reduced effectiveness against human players who could adapt to the agent's more predictable patterns. Conversely, more adaptive implementations showed higher random move rates and lower consistency, suggesting a core trade-off between strategic coherence and tactical flexibility.

These challenges point to fundamental limitations in current LLM-based approaches to game agents, suggesting the need for hybrid architectures that can better balance strategic consistency with adaptive gameplay. The persistence of these issues across different implementations indicates they may require structural solutions rather than merely prompt engineering or augmentation strategies.

6 Future Work and Research Directions

The limitations identified in our study point to several promising research directions that could significantly advance LLM-based game agents.

6.1 Hybrid Architecture Development

Our findings suggest that a pure LLM-centric approach may not be optimal for Pokémon battling. A hybrid architecture that combines the strengths of different AI approaches should be investigated. The LLM component would focus on high-level strategic reasoning and pattern recognition, while a reinforcement learning system with an evolving policy can handle adaptive strategy development. This hybrid approach would be augmented by specialized battle analysis tools for move validation and outcome prediction, creating a more robust decision-making pipeline.

6.2 Enhanced Decision Support Framework

To address the identified limitations in move selection and strategic planning, I propose developing a comprehensive decision support framework. At its core would be a centralized knowledge base providing quick access to type matchups, move effectiveness data, and common strategy patterns. This would be complemented by real-time battle analysis tools, including damage calculators and probability estimators for critical outcomes. A key component would be an enhanced memory system capable of tracking opponent behavior patterns and building a library of effective responses to common situations.

6.3 Improved State Representation

The challenges with state representation require a fundamental rethinking of how battle information is encoded and processed. Future work should explore compressed battle state encodings that maintain strategic information while reducing input length. This could be achieved through hierarchical state representations that separate immediate tactical information from longer-term strategic context. Additionally, developing dynamic state pruning mechanisms would allow the agent to focus on relevant information based on the current battle context.

6.4 Model Architecture Exploration

Our results suggest several promising directions for model architecture research. One approach would involve developing specialized small models fine-tuned specifically for battle state understanding. Another avenue would be exploring ensemble approaches that combine multiple specialized models for different aspects of gameplay. The integration of transformer-based architectures optimized for sequential decision-making could also provide significant improvements in strategic planning capabilities.

6.5 Evaluation Framework Enhancement

To better assess agent performance, future work should focus on developing more sophisticated evaluation methods. This includes creating comprehensive metrics that go beyond simple win rates to capture strategic sophistication and adaptation capabilities. Specialized benchmarks for testing

specific strategic capabilities would provide more granular insight into agent performance. Additionally, automated analysis tools could help identify decision-making patterns and failure modes, facilitating more targeted improvements to the system.

6.6 Integration and Deployment

The final phase of future work would focus on integrating these various components into a cohesive system. This would involve careful balancing of computational efficiency with strategic depth, ensuring the system remains practical for real-time battle situations. Performance optimization would be crucial, particularly in managing the interaction between different system components. The goal would be to create a battle agent that combines the strategic understanding of LLMs with the adaptability of reinforcement learning and the precision of specialized battle tools.

These research directions aim to address the core limitations identified in our current implementation while moving toward a more robust and adaptable battle agent. Success in these areas could have implications beyond Pokémon battles, potentially informing the development of LLM-based agents for other complex game environments.

7 Contribution Statement

This research was conducted independently by me, Garrett Herb. The project involved rebuilding the PokéLLMon implementation from the ground up using the Poke-env library as a foundation. All aspects of the work, including the system architecture design, implementation of various agent augmentations, experimental design, data collection, analysis, and manuscript preparation were carried out solely by the author. The code-base, experimental framework, and analytical tools were developed independently while building upon the concepts introduced in the original PokéLLMon paper.

Key contributions include:

- Complete re-implementation of the PokéLLMon system
- Development of novel agent augmentations
- Design and execution of all experiments
- Collection and analysis of battle statistics
- Preparation of the research manuscript and presentation materials

Here is a link to the code-base: https://github.com/gwherb/LLMagikarp

References

Bulbapedia. List of abilities. September 2024. https://bulbapedia.bulbagarden.net/wiki/Ability#List_of_Abilities

Bulbapedia. List of moves. September 2024. https://bulbapedia.bulbagarden.net/wiki/List_of_moves

Bulbapedia. List of Pokémon by National Pokédex number. September 2024. https://bulbapedia.bulbagarden.net/wiki/List_of_Pok%C3%A9mon_by_National_Pok%C3%A9dex_number

Bulbapedia. List of stats. September 2024. https://bulbapedia.bulbagarden.net/wiki/Stat

Costarelli, A., Allen, M., Hauksson, R., Sodunke, G., Hariharan, S., Cheng, C., Li, W., & Yadav, A. (2024). GameBench: Evaluating Strategic Reasoning Abilities of LLM Agents. *ArXiv*, abs/2406.06613.

Hao, S., Gu, Y., Ma, H., Hong, J.J., Wang, Z., Wang, D.Z., & Hu, Z. (2023). Reasoning with Language Model is Planning with World Model. *ArXiv*, abs/2305.14992.

Hu, J.E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ArXiv*, abs/2106.09685.

Hu, S., Huang, T., Ilhan, F., Tekin, S.F., Liu, G., Kompella, R.R., & Liu, L. (2024). A Survey on Large Language Model-Based Game Agents. *ArXiv*, abs/2404.02039.

Hu, S., Huang, T., & Liu, L. (2024). PokeLLMon: A Human-Parity Agent for Pokemon Battles with Large Language Models. *ArXiv*, abs/2402.01118.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *ArXiv*, abs/2005.11401.

Liu, S., Li, Y., Zhang, K., Cui, Z., Fang, W., Zheng, Y., Zheng, T., & Song, M. (2024). Odyssey: Empowering Agents with Open-World Skills. *ArXiv*, abs/2407.15325.

Shi, H., Xu, Z., Wang, H., Qin, W., Wang, W., Wang, Y., & Wang, H. (2024). Continual Learning of Large Language Models: A Comprehensive Survey. *ArXiv*, abs/2404.16789.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., & Wen, J. (2023). A Survey on Large Language Model based Autonomous Agents. *ArXiv*, abs/2308.11432.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ArXiv*, abs/2203.11171.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. *Trans. Mach. Learn. Res.*, 2024.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E.H., Xia, F., Le, Q., & Zhou, D. (2022). Chain of Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv*, abs/2201.11903.

Zhang, Y., Mao, S., Ge, T., Wang, X., Wynter, A.D., Xia, Y., Wu, W., Song, T., Lan, M., & Wei, F. (2024). LLM as a Mastermind: A Survey of Strategic Reasoning with Large Language Models. *ArXiv*, abs/2404.01230.